

Tex3D: Objects as Attack Surfaces via Adversarial 3D Textures for Vision-Language-Action Models

Anonymous Author(s)

Overview

This supplementary material provides essential details that complement our main paper. Sec. A delves into the effectiveness of our proposed modules, such as vertex-based texture parameterization and keyframe optimization, to clarify their specific contributions. Sec. B presents the complete pseudocode for the Tex3D optimization algorithm. Sec. C validates our trajectory-aware optimization strategy through an ablation study on keyframe weight reversal. Sec. D expands our literature review, detailing the current research landscape of 3D robustness in embodied AI, related work on 3D adversarial attacks, as well as simulation environments and differentiable rendering technologies. Sec. E provides extended quantitative analysis regarding cross-model transferability. Sec. F focuses on the impact of adversarial textures on color-dependent tasks within the LIBERO benchmark. Finally, Sec. G provides qualitative visualization results of the robot behavior before and after the attack.

A Further Analysis of Tex3D’s Core Components

This section expands upon the quantitative ablations presented in the main text by providing a mechanistic analysis of Tex3D’s two core design choices: *vertex-based texture parameterization* and *Trajectory-Aware Adversarial Optimization (TAAO)*. We detail the underlying reasons these components are structurally suited for 3D embodied attacks—specifically, how they regularize the adversarial optimization space and align gradient updates with the temporal dynamics of closed-loop control in complex embodied environments.

A.1 Mechanisms of Vertex-Based Texture Parameterization

A natural alternative to our formulation is to directly optimize the UV texture map in pixel space. However, existing graphics literature shows that standard UV-based texturing has several well-known representation issues, including seams, distortions, wasted UV space, vertex duplication, and varying resolution over the surface, especially when automatic UV unwrapping is used [7]. In addition, prior work on texture transfer and synthesis shows that reducing UV distortion often requires decomposing the surface into multiple charts, and that learning aligned UV maps across different shapes is itself a nontrivial problem [4]. Therefore, directly optimizing appearance in UV texture space is not entirely neutral with respect to the underlying surface parameterization.

By contrast, vertex-based color parameterization is defined directly on mesh vertices and does not rely on an explicit UV layout. As a result, it avoids UV-specific issues such as seam discontinuities and layout-dependent distortions during optimization. Moreover, vertex colors are rendered through interpolation over adjacent mesh faces, yielding a smoother, more surface-tied appearance representation than optimizing independent UV texels [2]. Our claim here is therefore not that vertex colors are more expressive than UV

textures, but that they define a more geometry-tied, interpolation-driven optimization space for adversarial appearance learning.

Under this view, the benefit of vertex parameterization in Tex3D is best understood not as stronger texture expressiveness, but as a simpler and smoother attack space defined directly on the 3D surface. For embodied attacks, where the perturbation must remain attached to the manipulated object across changing viewpoints, such a surface-based and interpolated parameterization is well suited to stable appearance optimization.

In summary, we use vertex colors not because they are more expressive than UV textures, but because they avoid several known issues of UV-based parameterization during optimization and provide a surface-tied representation whose interpolated structure naturally favors smoother appearance variations.

A.2 Trajectory-Aware Adversarial Optimization Improves Attack Robustness

The benefit of TAAO comes from better matching the optimization objective to the temporal structure of embodied control processes. Prior work on adversarial video generation has consistently shown that temporal information is important for effective video attacks. For example, Motion-Excited Sampler shows that simply transferring attack methods from image models to video models is less effective and that motion plays a key role in video attacking [33]. Likewise, Breaking Temporal Consistency explicitly incorporates temporal information into video attacks and improves attack effectiveness by manipulating temporal relations between neighboring frames in a structured manner [15]. Related work on sparse video attacks also suggests that strong adversarial influence need not be distributed uniformly across all frames, and can instead be concentrated on a subset of temporally critical frames [22, 28].

This temporal issue is even more important in embodied settings. Unlike passive video recognition, a VLA policy operates in closed loop, where each action directly affects future observations. Prior work on adversarial attacks against sequential decision-makers therefore argues that effective perturbations should respect environment dynamics rather than treat observations independently at each step [8]. Similarly, attacks on diffusion policies further emphasize that success requires continuously misleading the policy over time, rather than only fooling a single inference step in isolation [3].

From this perspective, TAAO can be understood as reallocating optimization effort toward temporally important parts of the trajectory, instead of spreading updates uniformly across all observations. This mechanism provides a plausible explanation for why TAAO outperforms uniform and random weighting in our ablations.

B Algorithm Pseudocode

Algorithm 1 summarizes the end-to-end optimization process of Tex3D. The pipeline is organized into two stages, namely trajectory collection and differentiable texture optimization, to explicitly

respect the inherently non-differentiable nature of standard embodied simulators while preserving the overall optimization objective defined over the full manipulation trajectory.

In the first stage, Tex3D runs the clean simulation environment to collect a reference trajectory and the corresponding rendering context at each timestep. Specifically, the clean observation is encoded by a pre-trained visual encoder to obtain latent features, from which latent velocity and acceleration are computed via central differences. After Min-Max normalization, a temperature-scaled softmax is applied to derive trajectory-aware frame weights, so that behaviorally critical transitions receive larger optimization emphasis. In parallel, the geometric parameters ($\mathbf{P}_t, \mathbf{V}_t, \mathbf{M}_t$) and lighting parameters (I_a, I_d, ρ) are relayed across renderers, and clean reference actions are recorded for the sampled views.

In the second stage, Tex3D performs differentiable adversarial appearance optimization through Foreground-Background Decoupling (FBD). At each timestep and sampled view, the target object is rendered in Nvdiffrast as a differentiable foreground, while the corresponding background is retained from MuJoCo. The two components are then composited into an adversarial observation, which is fed into the frozen VLA model to evaluate the adversarial objective. The resulting loss is first aggregated across sampled views and then reweighted by the pre-computed trajectory-aware coefficients, thereby encouraging optimization to focus on temporally important states along the manipulation sequence. The adversarial appearance is parameterized as learnable per-vertex colors, which are updated by gradient-based optimization and finally baked into a standard UV texture map for digital or physical deployment.

C Analysis of Keyframe Weight Reversal

To rigorously validate the necessity of our Trajectory-Aware Adversarial Optimization (TAAO) and its focus on behaviorally critical transitions, we conduct an ablation study by reversing the dynamically computed keyframe weights. Specifically, in the “Reversed Weights” setting, frames with high latent dynamics (which normally receive the highest optimization focus) are penalized with the lowest weights, while stable, non-critical frames are prioritized.

As shown in Table 1, applying the reversed weighting strategy significantly degrades the attack effectiveness across all four VLA models compared to our proposed dynamic weighting. This stark contrast confirms that the success of Tex3D heavily relies on concentrating adversarial perturbations on the most decision-sensitive moments (e.g., grasping or releasing) rather than uniformly or incorrectly distributing the attack budget across the entire trajectory.

D 3D Attacks and Differentiable Simulation

Building upon the discussion in the main paper, we further review the landscape of physical 3D adversarial attacks and the simulation technologies that enable their deployment in dynamic environments. Specifically, we survey existing 3D adversarial attacks in standard computer vision tasks and examine the current research landscape of 3D robustness in embodied AI. Furthermore, we discuss the critical role of differentiable rendering in bridging visual optimization and closed-loop physical evaluation in real settings.

Algorithm 1 Tex3D: Trajectory-Aware Adversarial 3D Texture Optimization

Input: VLA model π , Language instruction l , Target object mesh $\mathcal{M} = (\mathbf{V}, \mathbf{F})$, Simulator $\mathcal{S}$, Encoder E , Max iterations K , Learning rate η , Bound ϵ , Temperature τ .

Output: Adversarial texture map $\mathbf{T}_{adv}$

```

1: Phase 1: Trajectory Collection and Dynamics Extraction
2: Initialize simulator  $\mathcal{S}$  with state  $s_0$ .
3: for  $t = 1$  to  $T$  do
4:   Obtain observation  $\mathbf{O}_t$ , background  $\mathbf{x}_t^{bg}$ , and parameters  $(\mathbf{M}_t, \mathbf{V}_t, \mathbf{P}_t)$ .
5:   Predict clean action  $\mathbf{a}_t^* = \pi(\mathbf{O}_t, l)$ .
6:   Extract latent feature  $\mathbf{f}_t = E(\mathbf{O}_t)$ .
7: end for
8: for  $t = 2$  to  $T - 1$  do
9:   Compute velocity  $v_t$  and acceleration  $a_t$  via central differences.
10: end for
11: Normalize  $\hat{v}_t, \hat{a}_t$  over the trajectory and compute weights  $w_t = \text{Softmax}(\max(\hat{v}_t, \hat{a}_t)/\tau)$ .
12: Phase 2: Differentiable Texture Optimization
13: Initialize vertex colors  $\mathbf{c}$ .
14: for  $k = 1$  to  $K$  do
15:    $\mathcal{L}_{total} \leftarrow 0$ 
16:   for  $t = 1$  to  $T$  do
17:     if foreground mask  $\mathbf{m}_t$  is non-empty then
18:       Render  $\mathbf{x}_t^{fg}(\mathbf{c})$  and composite  $\mathbf{O}_t^{adv} = \mathbf{m}_t \odot \mathbf{x}_t^{fg} + (1 - \mathbf{m}_t) \odot \mathbf{x}_t^{bg}$ .
19:        $\mathcal{L}_{total} \leftarrow \mathcal{L}_{total} + w_t \cdot \mathcal{L}_{atk}(\pi(\mathbf{O}_t^{adv}, l), \mathbf{a}_t^*)$ .
20:     end if
21:   end for
22:   Update  $\mathbf{c}$  via Adam on  $\nabla_{\mathbf{c}} \mathcal{L}_{total}$ .
23: end for
24:  $\mathbf{c}^* = \tanh(\mathbf{c}) \cdot \epsilon$ 
25: Bake  $\mathbf{c}^*$  into  $\mathbf{T}_{adv}$ 
26: return  $\mathbf{T}_{adv}$ 

```

Table 1: Task failure rates (%) comparison between the proposed dynamic weighting and its reversed counterpart across four VLA models. Higher values indicate a more effective attack.

Model	Weighting Strategy	Spatial	Object	Goal	Long	Avg.
OpenVLA	Reversed Weights	83.2 (↑67.6)	74.1 (↑62.3)	75.4 (↑51.8)	81.6 (↑36.2)	78.6 (↑54.5)
	Dynamic Weights (Ours)	96.7 (↑81.1)	85.5 (↑73.7)	86.9 (↑63.3)	92.8 (↑47.4)	90.5 (↑66.4)
OpenVLA-OFT	Reversed Weights	68.5 (↑64.7)	64.2 (↑62.5)	67.8 (↑64.0)	69.1 (↑59.8)	67.4 (↑62.7)
	Dynamic Weights (Ours)	80.9 (↑77.1)	76.4 (↑74.7)	79.8 (↑76.0)	80.2 (↑70.9)	79.3 (↑74.6)
π_0	Reversed Weights	64.2 (↑60.7)	59.5 (↑57.2)	62.4 (↑57.2)	63.8 (↑56.6)	62.5 (↑57.9)
	Dynamic Weights (Ours)	75.9 (↑72.4)	70.2 (↑67.9)	73.4 (↑68.2)	73.7 (↑66.5)	73.3 (↑68.7)
$\pi_{0.5}$	Reversed Weights	61.5 (↑60.3)	57.8 (↑56.0)	60.1 (↑58.1)	61.7 (↑55.7)	60.3 (↑57.5)
	Dynamic Weights (Ours)	72.9 (↑71.7)	68.3 (↑66.5)	71.3 (↑69.3)	72.1 (↑66.1)	71.2 (↑68.4)

D.1 Traditional 3D Adversarial Attacks

Early studies on physically robust adversarial examples showed that adversarial perturbations can remain effective under viewpoint changes, camera noise, and transformations, and can even be instantiated as 3D-printed objects [1]. This line of work moved adversarial

attacks beyond purely digital image perturbations and established the feasibility of physically realizable adversarial objects.

Subsequent research greatly expanded the space of 3D attacks by directly manipulating richer 3D representations. One branch focuses on point-cloud attacks, where adversaries perturb point coordinates, add or remove points, or optimize the point set itself to fool 3D recognition models [23, 30]. In parallel, mesh-based attacks demonstrated that one can optimize surface geometry and texture through differentiable rendering, allowing adversarial perturbations to remain coherent across viewpoints while preserving a realistic 3D structure [32, 34]. Compared with planar 2D patches, these approaches are more naturally aligned with real-world physical scenes because the perturbation is defined directly on the 3D object.

More recently, adversarial attacks have been extended to neural scene representations such as Neural Radiance Fields. These methods exploit the inherent multi-view consistency of neural rendering to construct 3D-consistent adversarial objects and scenes with improved cross-view robustness [11, 13, 17]. Overall, existing 3D attacks cover a range of manipulable variables, including point structure, mesh geometry, texture, and neural volumetric representations [13, 23] across diverse settings and rendering conditions.

Despite this progress, most traditional 3D adversarial attacks are still developed for static, open-loop perception tasks such as image classification, 3D shape recognition, or object detection [17, 30, 32]. Their objective is typically to alter predictions from a fixed or pre-sampled set of viewpoints rather than to continuously influence an agent whose future observations depend on its current actions. As a result, although these methods provide an essential foundation for 3D-consistent perturbation generation, they do not address the closed-loop and temporally evolving nature of embodied decision-making over long-horizon interaction and control processes.

D.2 3D Robustness of Vision-Language-Action Models

Recent studies suggest that the robustness of VLA models is closely tied to their ability to handle physically grounded, three-dimensional variations in embodied environments. Rather than focusing only on conventional 2D image corruption, recent benchmarks and evaluations increasingly examine whether VLA policies remain reliable when the spatial configuration, viewpoint, object appearance, and other environment factors change during interaction.

Among existing studies, Eva-VLA provides the most direct evidence that 3D physical variation is a critical source of vulnerability for VLA systems. It evaluates VLA models under real-world physical variations and explicitly includes *3D object transformations* as one of its three core perturbation types, alongside illumination changes and adversarial patches [19]. In particular, Eva-VLA parameterizes 3D variation through object rotation and searches for worst-case physical conditions in a black-box manner, showing that even state-of-the-art VLA models can fail under seemingly natural changes in object pose and environment appearance [19]. These results indicate that VLA robustness should not be understood purely as resilience to pixel noise, but as the ability to maintain correct behavior under persistent changes in the physical 3D scene.

LIBERO-Plus further shows that current VLA models are highly sensitive to embodied perturbations closely related to 3D scene

understanding. Its robustness benchmark considers seven types of perturbations, including object layout changes, camera viewpoint changes, robot initial-state variations, lighting changes, background texture changes, language perturbations, and sensor noise [6]. Notably, the study reports substantial performance degradation under camera viewpoint shifts and robot initial-state changes, and also finds that policies are much more sensitive to target-object displacement than to irrelevant distractor objects [6]. This suggests that many VLA models still rely heavily on superficial spatial regularities and struggle to generalize when the embodied observation geometry changes in diverse real-world embodied scenarios.

A complementary perspective is provided by LIBERO-X, which introduces a hierarchical benchmark for evaluating VLA generalization under progressively more challenging distribution shifts [27]. Although LIBERO-X is not designed specifically as an adversarial benchmark, it explicitly incorporates spatial perturbations, scene topology reconstruction, and object attribute or texture variations [27]. These settings again highlight that embodied robustness is inherently entangled with 3D scene structure, object properties, and the spatial relationships that govern action execution in practice.

Taken together, these studies show that the robustness of VLA models is fundamentally constrained by their ability to cope with 3D and physically grounded variation. However, existing work mainly evaluates robustness through viewpoint changes, layout perturbations, object transformations, or patch-based modifications, while the vulnerability of VLA policies to adversarial perturbations embedded directly on object surfaces remains less explored. This gap motivates studying 3D-consistent, object-level perturbations that remain effective across changing viewpoints and throughout closed-loop interaction under long-horizon decision processes.

D.3 Simulation and Differentiable Rendering Pipelines

To study such attacks in a controlled yet interactive setting, researchers increasingly rely on simulation platforms for embodied AI. Several simulation ecosystems have become influential. MuJoCo is widely used for continuous control and robotic manipulation [26]. Isaac Gym emphasizes GPU-based large-scale physics simulation for robot learning [21]. SAPIEN and its ecosystem provide efficient simulation and rendering for articulated-object interaction and robot manipulation [10, 31]. Habitat is a major platform for embodied navigation and instruction-following in large-scale 3D indoor environments [25]. Recent surveys show these simulators play a central role in embodied navigation and manipulation research [29].

However, standard simulators are designed for forward simulation and rendering rather than end-to-end inverse optimization. This creates a mismatch for adversarial appearance optimization: the victim model consumes rendered observations, whereas the optimization target lies in the 3D object’s texture, material, or vertex attributes. Differentiable rendering addresses this gap by enabling a backward path from image-space objectives to 3D parameters.

Rasterization-based differentiable renderers such as nvdiffrast and PyTorch3D provide efficient gradients for mesh-based rendering and are well suited for optimizing texture maps, vertex colors, camera parameters, and related appearance variables [16, 24]. Soft

rasterization methods such as SoftRas and DIB-R relax the rendering pipeline to provide smoother gradients, especially for silhouette- and geometry-related optimization [2, 20]. Complementarily, physically based differentiable renderers such as redner and Mitsuba 3 support gradients with respect to richer scene parameters including geometry, materials, illumination, and volumetric effects, enabling higher-fidelity inverse rendering and scene optimization [14, 18].

In practice, many embodied attack pipelines couple a rigid-body simulator with a differentiable renderer. The simulator handles forward physical evolution, including rigid-body dynamics, contacts, and agent-environment interactions, while the differentiable renderer reconstructs the visual pathway required for appearance optimization [16, 32]. Under this architecture, gradients from the victim model can be backpropagated to the 3D asset, enabling optimization over texture maps, material parameters, or vertex colors without requiring the entire physics engine to be differentiable. This makes the simulator-renderer combination especially attractive for embodied attack research, where dynamic interaction is essential but the attack variable is primarily visual.

It is worth noting that differentiable physics and differentiable rendering are related but distinct. Differentiable physics engines such as Brax, Tiny Differentiable Simulator, and DiffTaichi are designed to backpropagate through system dynamics, contacts, and control trajectories [5, 9, 12]. By contrast, differentiable rendering focuses on gradients through the image formation process. For appearance-based attacks on embodied agents, differentiable rendering is often the more critical component, whereas full differentiability of the underlying dynamics may be unnecessary unless the attack explicitly optimizes trajectories, contacts, or object motion.

E Detailed Transferability Experiments

Expanding on the cross-model transferability results presented in Section 4.2 of the main paper, this section further provides the complete set of remaining detailed permutations of our transferability evaluation. Due to space constraints in the main text, we specifically focus here on the cross-family transferability when utilizing fine-tuned models (OpenVLA-OFT and $\pi_{0.5}$) as the source surrogates against target models from a different architectural family.

Transferability from Fine-Tuned Surrogates to Cross-Family Targets. As shown in Table 2, we evaluate the adversarial textures generated by OpenVLA-OFT against the π_0 family, and those generated by $\pi_{0.5}$ against the OpenVLA family, respectively.

A comparative analysis with the main text results reveals that utilizing a fine-tuned model as a surrogate can marginally enhance cross-architecture transferability compared to using its base counterpart. For instance, adversarial patterns crafted on OpenVLA-OFT and transferred to π_0 achieve a 45.1% failure rate on the Spatial task, which is higher than the 41.2% achieved when using the base OpenVLA as the surrogate (as reported in the main text).

A trend is observed when attacking OpenVLA using $\pi_{0.5}$ (54.5% failure rate) versus using the base π_0 (51.2% failure rate). This phenomenon suggests that the task-specific fine-tuning process, which adapts the VLA models to better capture domain-specific spatial and object-centric features, might align their gradient landscapes more closely across different architectures. Consequently, the adversarial patterns optimized on these fine-tuned representations

exhibit a slightly stronger capacity to generalize and disrupt the visual processing of unseen, cross-family target models, underscoring the broad threat posed by our vertex-based parameterization.

Table 2: Extended cross-model transferability results (%). This table presents the transfer combinations omitted from the main text, specifically focusing on fine-tuned surrogates attacking cross-family targets. Increases are relative to the target model’s clean performance.

Surrogate $\rightarrow$ Target	Spatial	Object	Goal	Long
OpenVLA-OFT $\rightarrow \pi_0$	45.1 ($\uparrow 41.6$)	38.6 ($\uparrow 36.3$)	42.4 ($\uparrow 37.2$)	48.7 ($\uparrow 41.5$)
OpenVLA-OFT $\rightarrow \pi_{0.5}$	35.8 ($\uparrow 34.6$)	31.2 ($\uparrow 29.4$)	34.7 ($\uparrow 32.7$)	40.5 ($\uparrow 34.5$)
$\pi_{0.5} \rightarrow$ OpenVLA	54.5 ($\uparrow 38.9$)	47.3 ($\uparrow 35.5$)	55.8 ($\uparrow 32.2$)	63.2 ($\uparrow 17.8$)
$\pi_{0.5} \rightarrow$ OpenVLA-OFT	46.2 ($\uparrow 42.4$)	39.8 ($\uparrow 38.1$)	44.6 ($\uparrow 40.8$)	51.5 ($\uparrow 42.2$)

F Impact of Color Keywords in Language Instructions

Although Tex3D perturbs object appearance through 3D texture modifications, one possible concern is that some task failures may stem from a mismatch between color-related words in the language instruction and the altered object appearance, rather than arising directly from disruption of the model’s underlying visuomotor processing itself. To more carefully examine this possibility, we conduct a controlled ablation study on the language interface.

Among the four LIBERO task suites, both the **Spatial** and **Long (LIBERO-10)** suites contain explicit color descriptors in their task instructions (e.g., “pick up the *black* bowl”, “put the *yellow and white* mug...”). Table 3 systematically lists all such instructions together with their corresponding color-removed counterparts. In this experiment, we remove these color-related terms while keeping the remaining components of each instruction strictly unchanged.

Table 4 compares task success rates under the original instructions and the color-removed instructions, for both clean (No Attack) and Tex3D settings. Across all four models, removing color keywords leads to only small and largely inconsistent changes in success rate, typically within ± 0.7 percentage points, regardless of whether the attack is applied. These results suggest that explicit color words are not a primary driver of task performance in these tasks. Therefore, the substantial degradation caused by Tex3D is more plausibly attributed to object-centric 3D texture perturbations interfering with the model’s visual-action mapping, rather than being mainly explained by a language-vision color mismatch.

G Visualization of Robot Behavior Before and After Attack on LIBERO

To illustrate how Tex3D disrupts closed-loop robot behavior, Fig. 1 compares clean rollouts (green borders) with attacked rollouts (red borders) under identical initial settings. While the clean VLA policy executes coherent manipulation sequences—from target localization to successful placement—the adversarial textures induce visible and persistent behavioral deviations. Rather than merely causing transient errors at isolated frames, the attack triggers small spatial deviations at behaviorally critical moments, such as pre-grasp alignment or contact establishment.



Figure 1: Qualitative visualizations of LIBERO task executions before and after the attack.

Table 3: Task instructions containing color or appearance keywords in the Spatial and Long (LIBERO-10) suites, with their color-removed variants. Color keywords are underlined.

Suite	Original Instruction	Color-Removed Instruction
Spatial	pick up the <u>black</u> bowl between the plate and the ramekin	pick up the bowl between the plate and the ramekin
	pick up the <u>black</u> bowl from table center and place it on the ...	pick up the bowl from table center and place it on the ...
	pick up the <u>black</u> bowl in the top drawer of the wooden cabinet	pick up the bowl in the top drawer of the cabinet
	pick up the <u>black</u> bowl next to the cookie box and place it on ...	pick up the bowl next to the cookie box and place it on ...
	pick up the <u>black</u> bowl next to the plate and place it on the ...	pick up the bowl next to the plate and place it on the ...
	pick up the <u>black</u> bowl next to the ramekin and place it on ...	pick up the bowl next to the ramekin and place it on ...
	pick up the <u>black</u> bowl on the cookie box and place it on ...	pick up the bowl on the cookie box and place it on ...
	pick up the <u>black</u> bowl on the ramekin and place it on the ...	pick up the bowl on the ramekin and place it on the ...
	pick up the <u>black</u> bowl on the stove and place it on the ...	pick up the bowl on the stove and place it on the ...
	pick up the <u>black</u> bowl on the <u>wooden</u> cabinet and place it on ...	pick up the bowl on the cabinet and place it on ...
Long (LIBERO-10)	put the <u>black</u> bowl in the bottom drawer of the cabinet	put the bowl in the bottom drawer of the cabinet
	put the <u>yellow</u> and <u>white</u> mug in the microwave	put the mug in the microwave
	put the <u>white</u> mug on the left plate	put the mug on the left plate
	put the <u>white</u> mug on the plate and push ...	put the mug on the plate and push ...

Table 4: Task success rates (%) on Spatial and Long (LIBERO-10) suites with original vs. color-removed language instructions in clean environments.

Model	Instruction Type	Spatial	Long
OpenVLA	Original	84.4	54.6
	Color-Removed	84.1	55.2
OpenVLA-OFT	Original	96.2	90.7
	Color-Removed	95.8	90.0
π_0	Original	96.5	92.8
	Color-Removed	96.2	92.5
$\pi_{0.5}$	Original	98.8	94.0
	Color-Removed	98.5	93.6

Due to the closed-loop nature of embodied control, these initial misalignments rapidly accumulate. As shown in the failure trajectories, this error propagation leads to compounding physical failures, including unstable grasping, misaligned reaching, or complete failure to maintain object contact. These visualizations directly corroborate the motivation behind our trajectory-aware formulation: temporally critical frames contribute disproportionately to task failure, highlighting the severe vulnerability of VLA policies to object-centric adversarial textures in long-horizon tasks.

References

- [1] Anish Athalye, Logan Engstrom, Andrew Ilyas, and Kevin Kwok. 2018. Synthesizing robust adversarial examples. In *International Conference on Machine Learning*. PMLR, 284–293.
- [2] Wenzheng Chen, Huan Ling, Jun Gao, Edward Smith, Jaakko Lehtinen, Alec Jacobson, and Sanja Fidler. 2019. Learning to predict 3D objects with an interpolation-based differentiable renderer. In *Advances in Neural Information Processing Systems*.
- [3] Yihong Chen, Yixiao Lin, Mingyu Yang, Zichuan Li, Chaoqi Wang, Chen Zhou, and Xiaojun Chang. 2024. Crafting Adversarial Attacks for Diffusion-Based Policies. In *Advances in Neural Information Processing Systems*.
- [4] Zhiqin Chen, Kangxue Yin, and Sanja Fidler. 2022. AUV-Net: Learning Aligned UV Maps for Texture Transfer and Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1465–1474.
- [5] Erwin Coumans and Yunfei Bai. 2021. Tiny Differentiable Simulator. GitHub repository.
- [6] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. 2025. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626* (2025).
- [7] Simone Foti, Stefanos Zafeiriou, and Tolga Birdal. 2024. UV-free Texture Generation with Denoising and Geodesic Heat Diffusion. In *Advances in Neural Information Processing Systems*.
- [8] Tim Franzmeyer, Philip H. S. Torr, and Chris Lu. 2024. Illusory Adversarial Attacks on Sequential Decision-Makers Using World Models. *Transactions on Machine Learning Research* (2024).
- [9] C. Daniel Freeman et al. 2021. Brax: A differentiable physics engine for large scale rigid body simulation. In *NeurIPS Datasets and Benchmarks*.
- [10] Jiayuan Gu, Fanbo Xiang, Xianzhi Li, Zhan Ling, Xuanlin Liu, Tongzhou Mu, Yuzhe Tang, Stone Tao, Xuanlin Wei, Yunchu Wang, Tony Z. Yuan, He Zhao, Yongxin Rao, Ziyi Huang, Yuke Zhu, Siyuan Huang, Yunzhu Ye, Sanqiang Chen, Tao Wang, Hao Yu, Yuan Wang, Chu Li, and Hao Su. 2023. ManiSkill2: A unified benchmark for generalizable manipulation skills. *arXiv preprint arXiv:2302.13100* (2023).
- [11] András Horváth and Csaba M. Józsa. 2023. Targeted adversarial attacks on generalizable neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*.
- [12] Yuanming Hu, Luke Anderson, Tzu-Mao Li, Qi Sun, Nathan Carr, Jonathan Ragan-Kelley, and Frédo Durand. 2020. DiffTaichi: Differentiable programming for physical simulation. In *International Conference on Learning Representations*.
- [13] Matthew Hull, Haoran Wang, Matthew Lau, Alec Helbling, Mansi Phute, Chao Zhang, Zsolt Kira, Willian Lunardi, Martin Andreoni, Wenke Lee, and Duen Horng Chau. 2025. RenderBender: A Survey on Adversarial Attacks Using Differentiable Rendering. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. 10473–10480.
- [14] Wenzel Jakob et al. 2022. Mitsuba 3. Project page and documentation.
- [15] Hee-Seon Kim, Minji Son, Minbeom Kim, Myung-Joon Kwon, and Changick Kim. 2023. Breaking Temporal Consistency: Generating Video Universal Adversarial Perturbations Using Image Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4325–4334.
- [16] Samuli Laine, Tero Karras, Jaakko Lehtinen, Timo Aila, and Tero Aittala. 2020. nvdiffrast. NVIDIA project page.
- [17] Leheng Li, Qing Lian, and Ying-Cong Chen. 2023. Adv3D: Generating 3D adversarial examples for 3D object detection in driving scenarios with Neural Radiance Fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*.
- [18] Tzu-Mao Li, Miika Aittala, Frédo Durand, and Jaakko Lehtinen. 2018. Differentiable Monte Carlo ray tracing through edge sampling. *ACM Transactions on Graphics* 37, 6 (2018), 1–11.
- [19] Hanqing Liu, Jiahuan Long, Junqi Wu, Jiacheng Hou, Huili Tang, Tingsong Jiang, Weien Zhou, and Wen Yao. 2025. Eva-VLA: Evaluating Vision-Language-Action Models' Robustness Under Real-World Physical Variations. *arXiv preprint arXiv:2509.18953* (2025).
- [20] Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. 2019. Soft rasterizer: A differentiable renderer for image-based 3D reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 7708–7717.
- [21] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. 2021. Isaac Gym: High performance GPU-based physics simulation for robot learning. In *NeurIPS Datasets and Benchmarks*.
- [22] Rui Mu, Lianli Wang, Qi Wang, Hu Zhang, Qiong Liu, and Nanning Zheng. 2024. Enhancing Robustness in Video Recognition Models: Sparse Adversarial Attacks and Beyond. *Neural Networks* 170 (2024), 313–324.
- [23] H. Naderi et al. 2023. Adversarial attacks and defenses on 3D point cloud classification: A survey. *Comput. Surveys* (2023).
- [24] Nikhila Ravi, Jeremy Reizenstein, David Novotny, Taylor Gordon, Wan-Yen Lo, Justin Johnson, and Georgia Gkioxari. 2020. Accelerating 3D deep learning with PyTorch3D. *arXiv preprint arXiv:2007.08501* (2020).
- [25] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. 2019. Habitat: A platform for embodied AI research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 9339–9347.
- [26] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems* (2012), 5026–5033.
- [27] Guodong Wang, Chenkai Zhang, Qingjie Liu, Jinjin Zhang, Jiancheng Cai, Junjie Liu, and Xinmin Liu. 2026. LIBERO-X: Robustness Litmus for Vision-Language-Action Models. *arXiv:2602.06556* [cs.CV] <https://arxiv.org/abs/2602.06556>
- [28] Zihan Wei, Yuhang Zhang, Ming Li, Shitong Wang, and Changsheng Xu. 2024. SVASTIN: Sparse Video Adversarial Attack via Spatio-Temporal Invertible Neural Networks. *arXiv preprint arXiv:2406.01894* (2024).
- [29] Lik Hang Kenny Wong, Xueyang Kang, Kaixin Bai, and Jianwei Zhang. 2025. A survey of robotic navigation and manipulation with physics simulators in the era of embodied AI. *arXiv preprint arXiv:2505.01458* (2025).
- [30] Chong Xiang, Charles R Qi, and Bo Li. 2019. Generating 3D adversarial point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 9136–9144.
- [31] Fanbo Xiang, Arsalan Mousavian, Clemens Eppner, and Dieter Fox. 2020. SAPIEN: A simulatable part-based interactive environment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11097–11107.
- [32] Chaowei Xiao, Dawei Yang, Bo Li, Jia Deng, and Mingyan Liu. 2019. MeshAdv: Adversarial meshes for visual recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6898–6907.
- [33] Hu Zhang, Linchao Zhu, Yi Zhu, and Yi Yang. 2020. Motion-Excited Sampler: Video Adversarial Attack with Sparked Prior. In *European Conference on Computer Vision*. Springer, 240–256.
- [34] Jingang Zhang et al. 2023. 3D adversarial attacks beyond point cloud. *Information Sciences* 637 (2023), 119046.